



PROG7311

Enterprise System Group Project
Part 1 — System Planning & Enterprise
Design

The V-Unit
Smart Retail System
Part 1 of 4
Talha, Zaheer, Yusuf, Lyle, Lindokuhle
ST10312198, ST10377352, ST10440377,
ST10255824, ST10083732
LU1 — Enterprise Thinking

1. System Overview

The Smart Retail System (SRS) is a bespoke, enterprise-grade retail management platform designed to unify and automate the core operations of a modern retail business. The system replaces the fragmented nature of traditional off-the-shelf retail software by integrating inventory management, point-of-sale (POS), customer relationship management, analytics, payment processing, notifications, reporting, and internal staff communication into a single, cohesive platform.

Retail organisations today face increasing pressure from e-commerce competition, rising customer expectations, supply chain complexity, and the need to make data-driven decisions in real time. The SRS is purpose-built to meet these demands by providing a scalable, reliable, maintainable, secure, and interoperable enterprise solution that can serve a single boutique store or a nationwide retail chain from the same architecture.

The system is designed with a microservices-oriented architecture, exposing RESTful APIs to support web, mobile, and third-party integrations. It is built with DevOps practices in mind — containerised with Docker, version-controlled via GitHub, and structured to support continuous integration and continuous delivery (CI/CD) pipelines — directly reflecting the change management and organisational growth principles covered in LU1.

2. Target Users

Enterprise software systems serve a broad, diverse user base with varying roles and needs. The SRS identifies the following primary and secondary stakeholders:

TABLE 1 — Target Users

User Role	Department	Primary System Interaction
Store Manager	Operations	Oversees daily operations, approves admin workflows, monitors analytics dashboard
Cashier / POS Operator	Sales	Processes sales transactions, applies discounts, handles customer returns
Inventory Clerk	Warehouse / Stock	Manages stock levels, processes deliveries, initiates purchase orders
Finance Officer	Finance	Accesses financial reports, reconciles transactions, generates exportable statements
Customer	External	Earns loyalty points, receives promotional notifications, views purchase history
System Administrator	IT / DevOps	Manages users, roles, permissions, system configuration and audit logs

3. System Scope

3.1 In Scope

- Inventory management with real-time stock tracking and low-stock alerts
- Point-of-sale (POS) module with barcode scanning and receipt generation
- Customer loyalty and rewards programme with points accumulation and redemption
- Real-time analytics and reporting dashboard (sales trends, revenue, stock velocity)
- Employee scheduling and attendance management
- Payment gateway integration supporting card, EFT, and QR code payments
- Event-driven push and email notification system for staff and customers
- Supplier and purchase order management
- Role-based access control (RBAC) and comprehensive audit logging
- Docker containerisation and GitHub-based collaborative version control

3.2 Out of Scope

- Physical hardware procurement (barcode scanners, POS terminals, receipt printers) — assumed pre-existing
- E-commerce storefront or online shopping cart — scoped for a future development phase
- Full payroll processing — finance module provides data export only
- Third-party ERP integration beyond defined API contracts in this phase

4. Enterprise Characteristics (Non-Functional Requirements)

LU1 defines non-functional requirements (NFRs) as the quality attributes that constrain how a system must behave, as opposed to what it must do. The following enterprise characteristics directly address the NFRs prescribed in sections 3.1–3.7 of the module manual.

TABLE 2 — Non-Functional Requirements (Enterprise Characteristics)			
NFR Category	LU1 Section	Attribute	Specification
Performance	S3.1	Response Time	API endpoints respond within 300 ms under normal load (< 500 concurrent users)
Performance	S3.1	Throughput	POS module supports minimum 100 transactions/minute per store during peak hours
Scalability	S3.2	Horizontal Scaling	Stateless microservices allow independent scaling of each module without architecture changes
Scalability	S3.2	Branch Capacity	System must support expansion to 50 branches without redesign
Reliability	S3.3	Uptime	Target 99.5% uptime; planned maintenance communicated 24 hours in advance
Reliability	S3.3	Graceful Degrad.	Non-critical service failure (e.g. notifications) must not affect core POS operations
Maintainability	S3.4	Code Quality	Minimum 70% automated test coverage; all changes reviewed via pull request before merge
Maintainability	S3.4	Modularity	Bounded contexts (inventory, sales, notifications) deployable and testable independently
Security	S3.5	Authentication	JWT-based auth; sessions expire after 30 minutes of inactivity
Security	S3.5	Data Protection	AES-256 encryption at rest; TLS 1.3 in transit; POPIA compliance for all customer PII
Usability	S3.6	Accessibility	UI complies with WCAG 2.1 AA standards; role-specific dashboards reduce cognitive load
Usability	S3.6	POS Speed	Cashiers must complete a standard sale in under 60 seconds
Interoperability	S3.7	API Standards	RESTful API with OpenAPI specification; all endpoints use standard HTTP verbs and JSON
Interoperability	S3.7	Data Export	CSV and Excel export for accounting software integration (Sage, Xero)

4.1 Performance (LU1 s3.1)

Performance defines how well the system responds under various load conditions. It is measured using metrics such as response time, throughput, and resource utilisation. Poor performance directly degrades the user experience and can result in lost sales at point-of-sale.

- All API endpoints must respond within 300 ms under normal load (< 500 concurrent users)
- The POS module must support a minimum of 100 transactions per minute per store during peak trading hours
- Analytics dashboard queries must complete within 2 seconds using pre-aggregated data sets
- Load testing will be conducted before each production release to validate performance thresholds

- Caching strategies (Redis) applied to high-read-frequency data such as product catalogues and pricing

4.2 Scalability (LU1 s3.2)

Scalability is the system's ability to handle increased workload without degrading performance or requiring architectural changes. LU1 identifies two approaches: vertical scaling (adding resources to existing nodes) and horizontal scaling (adding more nodes). The SRS is designed for horizontal scaling as it offers greater fault tolerance and cost efficiency.

- Stateless microservice design enables horizontal scaling of individual modules (e.g., analytics, notifications) independently
- Docker containerisation allows additional service instances to be spun up dynamically in response to demand
- Asynchronous communication (event queues) is preferred over synchronous calls to improve throughput under load
- Database read replicas distribute query load during peak reporting periods
- Cloud-native deployment supports auto-scaling on platforms such as AWS ECS or Azure Kubernetes Service
- System architecture must support expansion to 50 branches without changes to core service design

4.3 Reliability (LU1 s3.3)

Reliability is the ability of the system to perform its required functions correctly and consistently over time. Unreliable software damages customer trust and enterprise reputation. The SRS applies design-phase techniques to maximise reliability.

- Target system uptime of 99.5%; planned maintenance windows communicated 24 hours in advance
- Graceful degradation: if a non-critical service (e.g., notifications) fails, core POS operations continue unaffected
- Transaction atomicity enforced in the POS and payment modules to prevent partial state corruption
- Automated health checks and service restart policies configured in Docker Compose and container orchestration
- Comprehensive error handling and structured logging (using correlation IDs) to enable rapid fault diagnosis
- Regular automated backups of all persistent data stores with tested restoration procedures

4.4 Maintainability (LU1 s3.4)

Maintainability is the ease with which a system can be modified to fix defects, improve performance, or adapt to a changed environment. LU1 highlights spaghetti code as the antithesis of maintainability. The SRS avoids this through disciplined application of SOLID principles and design patterns.

- SOLID principles applied throughout: single responsibility per class/service, open for extension, closed for modification

- Repository pattern isolates data access logic from business logic, enabling database technology changes without service rewrites
- Modular bounded contexts (inventory, sales, notifications, reporting) can be updated independently without cross-module regression
- OpenAPI/Swagger documentation maintained for all REST endpoints, reducing developer onboarding time
- Minimum 70% automated test coverage enforced; all code changes reviewed via pull request on GitHub before merge
- Docker Compose standardises development, staging, and production environments, eliminating environment-specific defects

4.5 Security (LU1 s3.5)

Software security must be considered throughout the entire software development lifecycle, not only at the deployment phase. LU1 distinguishes between application security (protecting running software) and software security (building security in from the design stage). The SRS adopts both.

- JWT-based authentication with short-lived access tokens and secure refresh token rotation
- Role-based access control (RBAC) enforces the principle of least privilege — users access only what their role requires
- All data encrypted at rest (AES-256) and in transit (TLS 1.3)
- PCI-DSS awareness in payment module design — raw card data is never stored or processed directly by SRS servers
- Parameterised queries and input validation at all data entry points to prevent SQL injection and cross-site scripting (XSS)
- Audit log records all privileged operations with user ID, timestamp, IP address, and action detail; retained for 12 months
- POPIA compliance for all South African customer personal information (data residency within ZA jurisdiction)

4.6 Usability (LU1 s3.6)

Usability describes how effectively, efficiently, and satisfyingly users can interact with the system to achieve their goals. LU1 identifies user-centred design principles and the distinction between usability and utility. The SRS addresses both dimensions.

- POS interface optimised for speed — cashiers must be able to complete a standard sale in under 60 seconds
- UI must comply with WCAG 2.1 AA accessibility standards to support users with disabilities
- Role-specific dashboards present only the information and actions relevant to that user's context (reducing cognitive load)
- Consistent interface guidelines (typography, colour system, component library) applied across all modules
- Onboarding workflows and contextual help tooltips included for complex administrative tasks
- Usability testing planned with representative users from each role group during the prototype phase

4.7 Interoperability (LU1 s3.7)

Interoperability is the ability of the system to exchange information and work with external systems in a coordinated manner. LU1 references the Levels of Conceptual Interoperability Model (LCIM), which defines interoperability from technical connectivity through to conceptual alignment. The SRS is designed at technical and syntactic interoperability levels.

- RESTful API layer exposes all core domain operations using standardised HTTP verbs and JSON payloads
- Webhook support enables event-driven integrations with external systems (e.g., supplier portals notified on low stock)
- Payment gateway abstraction layer supports multiple providers (PayFast, Stripe, Ozow) behind a single interface
- CSV and Excel data export enables interoperability with accounting software (Sage, Xero) without API integration
- OpenAPI specification published so third-party developers can build integrations without access to source code
- API versioning strategy implemented to ensure interoperability is maintained through future SRS updates

5. Functional Requirements (LU1 §2.3)

Functional requirements define what the system must do — the specific behaviours and operations it must support. LU1 prescribes that good requirements should be clear, unambiguous, testable, prioritised, and traceable. Each requirement below follows the format: 'The system shall...'

TABLE 3 — Functional Requirements			
ID	Module	Priority	Requirement
FR-01	Authentication	High	The system shall authenticate users via username and password and issue a JWT access token upon successful login
FR-02	Inventory	High	The system shall track stock levels in real time and trigger configurable low-stock alerts to relevant staff
FR-03	POS	High	The system shall process sales transactions including barcode scanning, quantity selection, discount application, and receipt generation
FR-04	Analytics	High	The system shall display a real-time dashboard showing sales trends, top-selling products, and revenue summaries by period
FR-05	Notifications	Medium	The system shall deliver push and email notifications to staff and customers based on configurable system event triggers
FR-06	Reporting	Medium	The system shall generate scheduled and on-demand financial and operational reports exportable in PDF and CSV formats
FR-07	Admin Workflow	Medium	The system shall provide an admin module for user account management, role assignment, and system configuration
FR-08	Payments	High	The system shall integrate with external payment gateways to process card, EFT, and QR code payments via a provider-agnostic layer
FR-09	Messaging	Medium	The system shall provide an internal messaging component for direct staff-to-staff and broadcast management-to-team communication
FR-10	Loyalty	Low	The system shall manage customer loyalty profiles, accumulate reward points on purchase, and support point redemption at checkout
FR-11	Purchase Orders	Medium	The system shall allow inventory clerks to create and submit purchase orders to registered suppliers when stock falls below threshold
FR-12	Audit Log	High	The system shall log all privileged operations with user identity, timestamp, IP address, and a description of the action performed

6. Enterprise Considerations (LU1 s4)

6.1 Emergent Behaviour (LU1 s4.1)

Emergent behaviour refers to system behaviours that arise from the interaction of components and are not explicitly designed into any single component. In enterprise software, emergent behaviour can be desirable (e.g., improved system throughput from caching interactions) or undesirable (emergent misbehaviour), such as race conditions, cascading failures, or unexpected data states.

The SRS addresses emergent misbehaviour risks through the following design decisions:

- Transactional boundaries clearly defined in the POS and payment modules to prevent partial state corruption under concurrent load
- Event queue (pub/sub) architecture in the Notification Service decouples producers from consumers, preventing notification failures from blocking core transaction processing
- Circuit breaker patterns applied to external payment gateway calls to prevent a failing provider from cascading into POS unavailability
- Integration test suites simulate multi-module interaction scenarios to detect emergent behaviours before production deployment

6.2 Organisational Growth (LU1 s4.2)

LU1 identifies that growing organisations face a choice between bespoke software development and off-the-shelf solutions. Bespoke software offers competitive differentiation

and precise fit to business requirements, while off-the-shelf solutions offer faster time-to-value but limited customisation. The SRS is a bespoke solution justified by the following:

- Off-the-shelf retail platforms cannot be customised to integrate the specific loyalty, messaging, and analytics requirements defined by The V-Unit's system
- Bespoke development allows the team to maintain full control over data models, API contracts, and security policies
- The modular architecture supports team growth: new developers can be onboarded to a single bounded context without requiring full system knowledge

The development team will progress through the recognised stages of a growing software team across the four project parts: from initial planning (Part 1), through design and prototyping (Part 2), to full build (Part 3), and finally to DevOps integration and delivery (Part 4). Each stage introduces new coordination challenges that are managed through GitHub branching strategy and structured code reviews.

6.3 Change Management (LU1 s4.3)

Change management in software development governs how changes to requirements, code, and infrastructure are planned, reviewed, and deployed. LU1 emphasises the role of continuous integration and continuous delivery (CI/CD) in enabling effective change management in agile environments.

The SRS project applies the following change management practices:

- GitHub Flow branching strategy: all feature development occurs on named branches; changes are merged to main only via reviewed and approved pull requests
- Commit history on GitHub serves as the team's audit trail — all contributions are individually attributed (as required by the project brief)
- Continuous integration: automated test suites run on every pull request to detect regressions before code is merged
- Docker Compose used to standardise all environments (development, testing, staging, production), eliminating environment-specific change risk
- Semantic versioning applied to the API to classify changes as patches, minor additions, or breaking changes — enabling interoperability with dependent systems to be managed
- Standard changes (routine deployments) automated via CI/CD; significant architectural changes require team review and lecturer sign-off before implementation

7. Individual Functional Requirements

Each team member has contributed one functional requirement to the system design. Each requirement is described, justified in terms of business need, and contextualised within the enterprise architecture of the Smart Retail System. All requirements are beyond login and registration functionality, per the project brief.

TABLE 4 — Individual Functional Requirements (Per Member)			
Member	Requirement	Justification	Enterprise Fit
Talha	Analytics Dashboard — real-time sales trends, revenue summaries, best-selling products, filterable by date and branch	Managers need up-to-date BI to optimise stock and plan promotions; manual spreadsheets are error-prone and unscalable	Dedicated Analytics Service queries pre-aggregated schema; decouples read load from POS write DB. Supports Performance (S3.1) and Scalability (S3.2)
Zaheer	Payment Gateway Integration — provider-agnostic abstraction layer for card, EFT, and QR code payments	Single hard-coded provider creates vendor lock-in; PCI-DSS compliance requires raw card data never stored on SRS servers	Adapter pattern wraps each provider behind IPaymentProvider interface. Open/Closed Principle applied. Supports Security (S3.5) and Interoperability (S3.7)
Yusuf	Notifications Module — event-driven push and email notifications to staff and customers on configurable system triggers	Retail is time-sensitive; staff need immediate alerts for low stock and failed payments; manual comms are unreliable at scale	Observer/pub-sub pattern; decoupled from POS so notification failures cannot block transactions. Supports Reliability (S3.3) and Scalability (S3.2)
Lyle	Reporting System — scheduled and on-demand PDF/CSV reports for financial, inventory, and operational data	Audit obligations and financial reviews require automated, consistent reports across all branches without manual effort	Async background job queries read-replica DB to isolate reporting load. Scheduled via job scheduler. Supports Performance (S3.1) and Maintainability (S3.4)
Lindokuhle	Internal Messaging — direct staff-to-staff and broadcast management-to-team messaging with persistence and read receipts	Personal platforms (WhatsApp) create audit gaps and data leakage risks; internal system ensures traceable, role-governed communication	WebSocket real-time delivery with REST fallback. RBAC controls broadcast vs direct. All messages in audit log (FR-12). Supports Security (S3.5) and Maintainability (S3.4)

8. Project Roadmap & Learning Unit Alignment

The Smart Retail System is developed across four project parts, each aligned to specific learning units in PROG7311. The table below maps the project phases to the module learning outcomes.

TABLE 5 — Project Roadmap & Learning Unit Alignment			
Part	Learning Unit	Focus	SRS Deliverable
Part 1	LU1 — Enterprise Thinking	System planning, NFRs, enterprise characteristics	System overview, target users, scope, NFRs (Performance, Scalability, Reliability, Maintainability, Security, Usability, Interoperability), Emergent Behaviour, Organisational Growth, Change Management, Individual Requirements
Part 2	LU2 — Patterns & Architecture	Design patterns, UML, API contracts, mockups	UML class & sequence diagrams, design patterns (Repository, Observer, Adapter, Factory), OpenAPI specification, UI wireframes and mockups
Part 3	LU3 + LU4 — Build & Optimise	Full implementation and performance optimisation	All modules implemented, database schema, REST API, security hardening, performance profiling, caching strategies, load testing
Part 4	LU5 — Methodologies & DevOps	CI/CD, Docker, final testing, presentation	Docker containerisation, Docker Compose multi-service config, GitHub Actions CI/CD pipeline, final integration testing, team presentation

8.1 LU1 Coverage Checklist

The following table confirms coverage of all LU1 learning objectives within this Part 1 document:

TABLE 6 — LU1 Coverage Checklist		
LU1 Learning Objective	Module Manual Section	Covered In
Explain the scope of enterprise software systems	S1, S2.1	Section 1 & 2
Differentiate between functional and non-functional requirements	S2.3, S3	Section 4 & 5
Explain how non-functional requirements impact software design	S3.1–3.7	Section 4
Discuss emergent behaviour	S4.1	Section 6.1
Discuss challenges of organisational growth in enterprise systems	S4.2	Section 6.2
Motivate the need for change management in large enterprises	S4.3	Section 6.3